

RELAZIONE PROGETTO: COMPANION TRAVEL

Relazione Progetto del corso di Laboratorio di Applicazioni Mobili denominato Travel Companion a cura dello studente Nicola Tinari: matricola 1078643 e indirizzo email nicola.tinari@studio.unibo.it .

1. INTRO

Travel Companion è un'applicazione mobile per dispositivi Android, sviluppata per assistere l'utente nella pianificazione, registrazione e consultazione dei propri viaggi. L'app consente di:

- Creare piani di viaggio indicando la data di inizio e di fine, e i luoghi di partenza e arrivo.
- Avviare e interrompere manualmente il tracciamento GPS.
- Documentare il viaggio aggiungendo note e fotografie in qualsiasi momento.
- Visualizzare lo storico dei viaggi effettuati, con possibilità di filtrare per tipologia e periodo.
- Consultare grafici e statistiche relative ai percorsi registrati.
- Visualizzare su mappa i tracciati dei viaggi.

L'applicazione è stata progettata per offrire un'esperienza d'uso semplice, intuitiva e completa, con l'obiettivo di fornire tutte le informazioni utili per la consultazione e l'analisi dei viaggi passati. Inoltre la grafica è stata progettata in maniera tale da garantire all'utente un'interfaccia semplice, ma che allo stesso tempo non tralasciasse nessun dettaglio rilevante. Per testare tutte le sue funzionalità, oltre al classico emulatore di android studio, è stato usato un Huawei p30Lite (android 10) e per effettuare alcuni cambi di posizione è stata usata anche l'applicazione FakeGPS

2. DATABASE E ARCHITETTURA

Composizione del database

Prima di passare alla creazione dell'applicazione con le varie schermate, azioni, servizi e notifiche, la prima parte del lavoro ha riguardato la definizione del **database** su cui l'applicazione si appoggia. L'applicazione utilizza **Room**, la libreria di persistenza ufficiale di Google, come livello di astrazione su SQLite.

La base di dati è composta dalle seguenti entità principali:

- **Trip** -> ovvero un singolo viaggio avente un id univoco, un luogo di partenza, un luogo di arrivo, una data di partenza (formato yyyy-MM-dd), una data di arrivo e identificato da una variabile TripType una classe enum in cui sono scritti tutti i tipi di viaggi possibili:

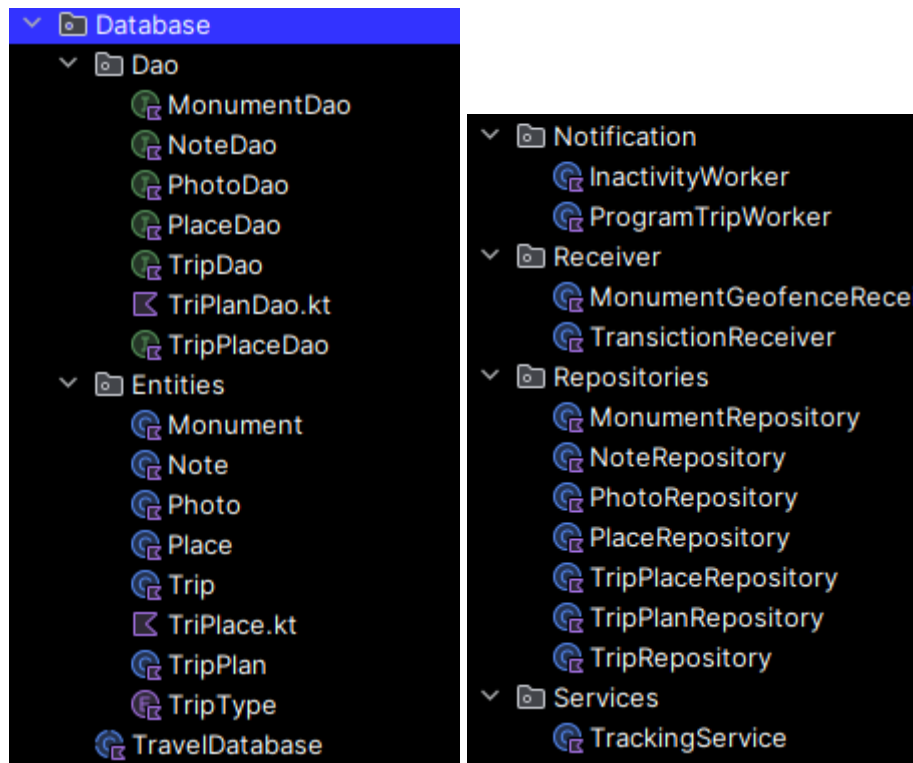
- Local: usato per identificare i viaggi più brevi di un giorno o mezza giornata per la propria città e pensata per i viaggi a piedi o in bici più che per i viaggi in macchina
- Excursion: usato per i identificare i viaggi fuori città come appunto un escursione e anche questi pensati per durare massimo un giorno
- Journey: usato invece per le vacanze di più giorni dove l'utente non si limita ad una semplice passeggiata per la città, ma alla visita di più città (ad esempio visitare tutta la riviera romagnola)
- **Place** -> entità che rappresenta un luogo visitato identificato anch'esso da un id univoco, coordinate di latitudine, coordinate di longitudine (entrambe arrotondate a 4 cifre dopo la virgola che nella realtà una variazione di uno di essi corrisponde ad una variazione di circa 10 metri) e un nome per rendere la visualizzazione dei luoghi più semplice all'utente.
- **Photo** -> entità che rappresenta una singola fotografia importata dalla fotocamera e associata ad un luogo (place) durante un particolare viaggio (trip). Essa è identificata quindi dall'id_trip (foreign key di Trip.id), da un id_place (foreign key di [Place.id](#)), un time_stamp così da poter ordinare le varie fotografie in base a quando sono state scattate e un image_path (stringa) per poterle andare a riprendere nella galleria. La chiave primaria sono id_place e image_path, dato che è impossibile associare allo stesso place (luogo) la stessa foto. Inizialmente si era pensato di memorizzare le foto attraverso la loro serializzazione (blob), ma in seguito a vari test e al limite di caratteri che possono essere inseriti in una cella del database è stata scartata come opzione.
- **Note** -> questa entità rappresenta una delle note scritte dall'utente durante un viaggio, identificata anch'essa da un id ma associata ad un trip e ad un place anch'essa come chiave primaria.
- **TriPlace** -> non è proprio un elemento ma bensì una relazione che associa ad un trip i vari places aggiungendo anche un riferimento al momento in cui sono stati registrati così da poterli avere sempre ordinata all'occorrenza.
- **TripPlan** -> entità in cui vengono memorizzati tutti i viaggi programmati dall'utente, anche quelli che alla fine non saranno mai realmente compiuti
- **Monument** -> entità dove vengono memorizzati alcuni monumenti o luoghi importanti per cui l'utente vuole ricevere notifiche quando si trova in prossimità di uno di essi.

Gestione delle operazioni

Le query di accesso ai dati sono state incapsulate nei **DAO**. Ogni entità e relazione ha il proprio DAO: TripDao, PlaceDao, PhotoDao, NoteDao, TriPlanDao, MonumentsDao e TripPlaceDao.

La classe TravelDatabase è definita come **singleton**, in modo che sia presente una sola istanza nel ciclo di vita dell'applicazione. Le uniche classi autorizzate a interagire direttamente con il database sono i **Repository**, uno per ciascuna entità (TripRepository, PlaceRepository, TripPlaceRepository, NoteRepository, TriPlanRepository,

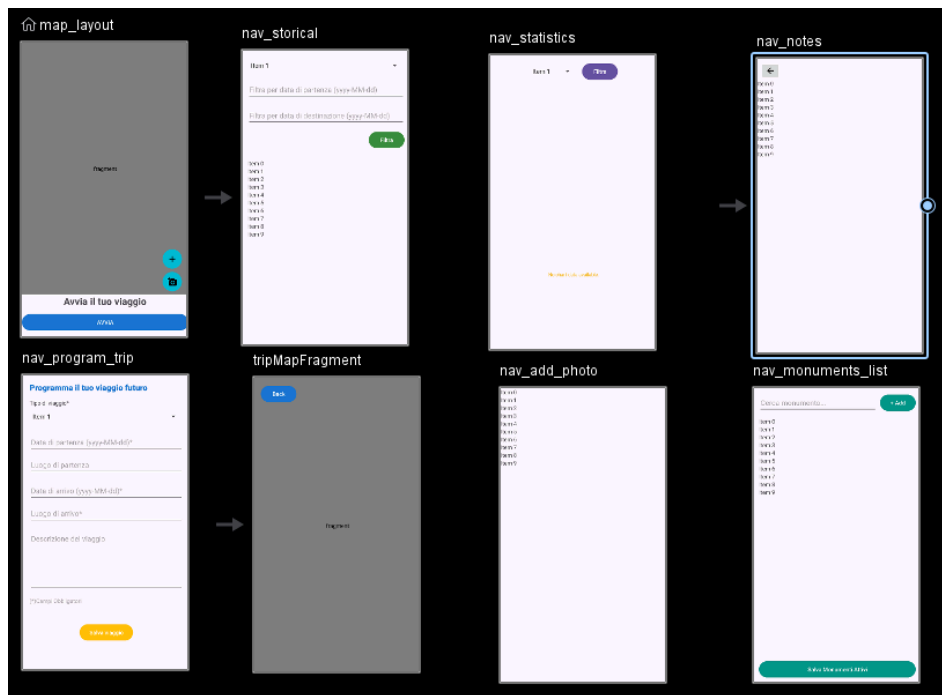
MonumentsRepository e PhotoRepository). Ogni Repository riceve in input l'istanza del database e utilizza le query definite nei rispettivi DAO. Questo approccio ci permette separare in modo netto la logica di accesso ai dati dalla logica di presentazione (rispettando il pattern MVVM), favorendo la manutenibilità e la flessibilità. A differenza degli altri pattern MVVM, oltre ad avere una logica molto intuitiva, ci permette di separare la parte visiva da quella logica e dalle entità, così non solo da avere un codice pulito rispettando alcuni dei principi fondamentali dell'ingegneria della progettazione object oriented (ovvero Dependency Inversion Principle e il Single Responsibility Principle), ma anche di rendere più semplici al programmatore le operazioni di debug.



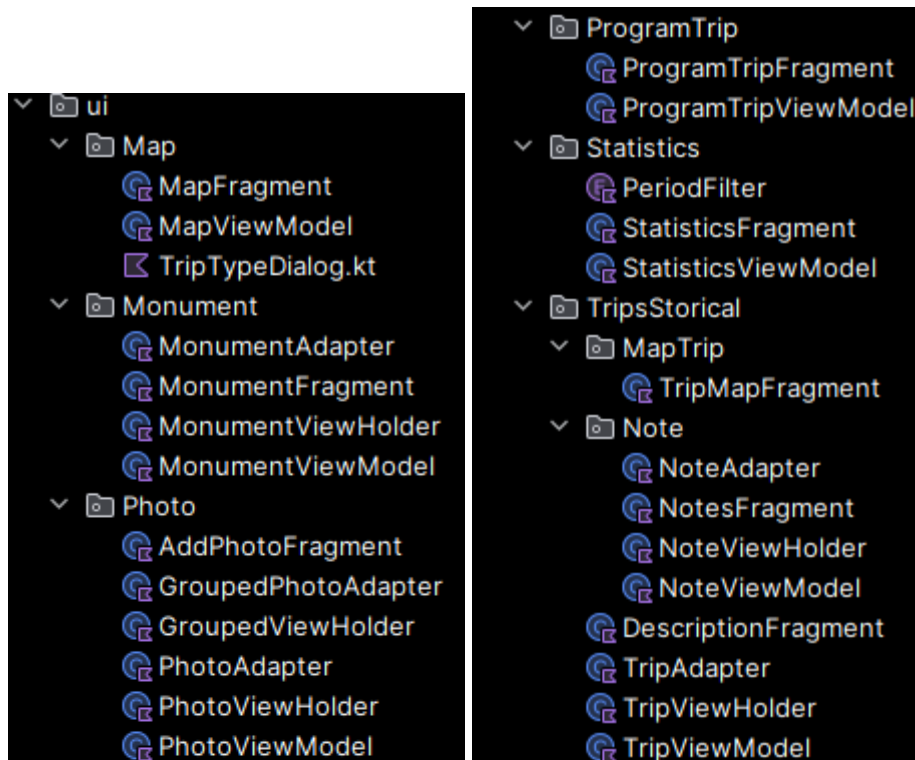
3. Struttura dell'applicazione

Panoramica del Navigation

Adesso passiamo alla descrizione delle varie schermate e componenti dell'applicazione. Ho deciso di strutturare l'applicazione utilizzando il Navigation Component con un *navigation graph*, che consente di gestire il passaggio tra schermate mediante Fragment. Questa scelta è motivata dalla leggerezza dei Fragment e dalla loro semplicità di gestione. Nella figura seguente è riportato uno screenshot del **navigation graph** dell'applicazione, realizzato mediante l'editor visuale di Android Studio. Ogni destinazione corrisponde a un **Fragment** dedicato, mentre le frecce indicano le azioni di navigazione configurate. Questo approccio permette di gestire in modo chiaro e intuitivo la navigazione tra le varie schermate, riducendo la complessità del codice e migliorando la coerenza dell'esperienza utente.



Per migliorare l'organizzazione del progetto, tutte le schermate sono state raccolte nel package ui. All'interno di esso ho creato sotto-package dedicati per ciascun modulo funzionale (ad esempio Statistics, Map, ProgramTrip).

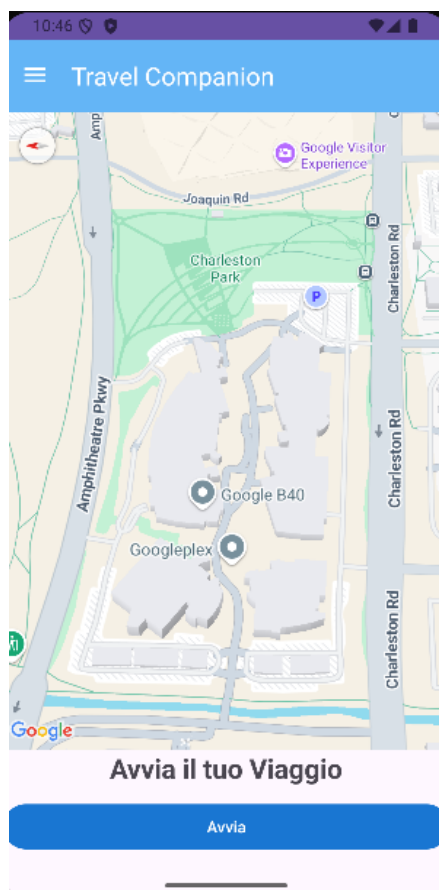


Ogni schermata è associata a un ViewModel che gestisce la logica di presentazione e l'accesso ai dati (LiveData) e un Fragment che osserva i dati e li presenta. Fanno eccezione i moduli Photo e TripsStorical, che sono principalmente dedicati alla visualizzazione e per cui ho previsto l'uso di ViewHolder(essendo moduli dedicati alla visualizzazione di liste tramite recycler view) e Adapter (che gestisce alcune operazioni di filtraggio per motivi di semplicità). In questi moduli, il ViewModel espone i dati come LiveData, mentre l'Adapter e il ViewHolder si limitano alla presentazione. In particolare è stato creato: un Adapter per la gestione della singola fotografia; un Adapter per la gestione di una lista di fotografie; un Adapter per l'elenco delle note associate a un viaggio e un Adapter per l'elenco di tutti monumenti registrati dall'utente.

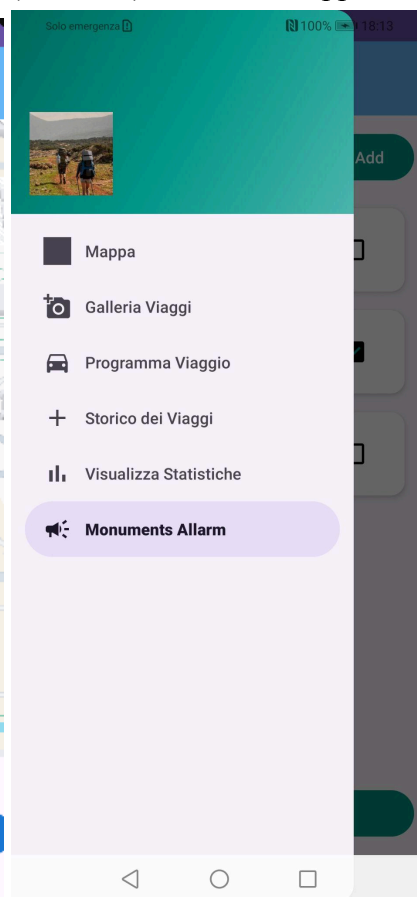
Schermate Principali

Appena avviata l'applicazione saremo indirizzati a una schermata (MapFragment) che rappresenta la schermata principale dell'applicazione in cui viene svolta la funzionalità principale della nostra applicazione: l'avvio manuale di un viaggio.

Da questa schermata possiamo subito vedere una toolbar celeste dalla quale siamo in grado di accedere alle schermate principali (non tutte) della nostra applicazione.



(map_layout.xml)



(activity_main.xml)

1. **Map** -> ovvero la prima schermata in cui si trova l'utente appena entrato nell'applicazione, in cui sarà possibile avviare il tracciamento ed interromperlo in maniera

manuale, aggiungere foto e note e il percorso sarà monitorato direttamente sulla mappa tramite polyLine

2. **Photo** -> Un recycler view dove per ogni viaggio (avente almeno una foto) saranno riportate tutte le foto raccolte dall'utente durante il suo svolgimento
3. **ProgramTrip** -> Un form in cui un utente sarà in grado di programmare viaggi futuri specificando il tipo, la partenza, la destinazione e l'arco temporale. Ovviamente sono implementati anche i vari meccanismi di controlli (es: inserire la data in un formato non congruo a quello indicato)
4. **Trips Storical** -> un recycler view in cui saranno scritte le informazioni principale del viaggio (tipo, arco temporale, distanza percorsa ...) e sarà possibile accedere anche ad altre due schermate (non accessibili tramite toolbar)
 - a. **Note** -> un altro recycler view dove sono presenti le note scritte dall'utente durante il viaggio e il time_stamp in cui sono state scritte
 - b. **MapTrip** -> un ulteriore mappa in cui è possibile visualizzare il percorso effettuato sempre tramite polyLine rossa questa volta. Questo Fragment non ha un view model tutto suo semplicemente perché per la poca logica di cui necessitava (un singolo metodo) si è preferito riciclare la classe TripViewModel
5. **Statistics** -> una schermata in cui sono presenti una HeatMap, un grafico a barre e un linechart (relativo alle distanze percorse) e anche questi viaggi sono filtrabili in base a 4 archi temporali: 1_MONTH, 3_MONTHS, 8_MONTHS e ALL.
6. **Monuments Allarm** -> una schermata in cui troviamo una lista di monumenti, tramite add si possono aggiungere di nuovi, per avviare il sistema di geofence basta premere il bottone salva ed è possibile filtrare i monumenti con una barra di ricerca

4. Dettagli di schermate e funzionalità

Facciamo adesso un piccolo approfondimento sugli elementi principali su cui si fonda un' applicazione android ovvero il MainActivity e il Manifest.

4.1 AndroidManifest

Il file AndroidManifest.xml definisce i permessi, le componenti principali (activity, servizi, receiver) e le configurazioni globali dell'applicazione.

L'applicazione richiede i seguenti permessi per garantire il corretto funzionamento delle funzionalità di tracking, notifiche e riconoscimento attività: ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, ACCESS_BACKGROUND_LOCATION, INTERNET, POST_NOTIFICATIONS, ACTIVITY_RECOGNITION e FOREGROUND_SERVICE_LOCATION.

In questo file xml vengono anche dichiarati i componenti come: MainActivity, TrackingService, TransictionReceiver e anche l'integrazione con l'API_KEY di google maps.

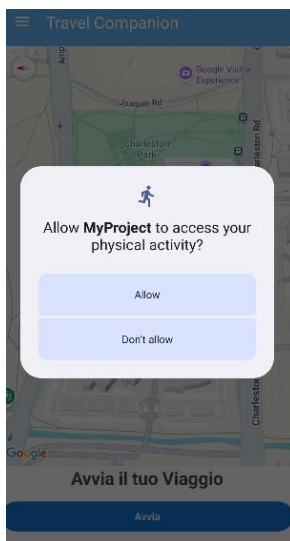
4.2 MainActivity

Il cuore della nostra applicazione dove viene inizializzato il Navigation Drawer, i worker periodici, sono richiesti i permessi in run time e viene avviato il riconoscimento di attività

(quest'ultimo serve per rilevare se l'utente si sta muovendo).

La configurazione del navigation Drawer avviene nel metodo onCreate e si inizia inizializzando contenitore principale, barra superiore (toolBar), il menù laterale e il navHostFragmen (layout_navigation.xml). Poi si procede configurando il toggle (icona ad hamburger) con le azioni di apertura e chiusura del menù e impostando il contenitore. Infine vengono settati il listener per la selezione delle voci e viene effettuato il collegamento con la UI.

Il metodo requestAllPermissions() verifica progressivamente i permessi di ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, ACCESS_BACKGROUND_LOCATION, POST_NOTIFICATIONS, ACTIVITY_RECOGNITION. Nel caso in cui uno di loro non dovesse essere concesso allora viene richiesto e l'esito gestito in onRequestPermissionsResult().



Il metodo startActivityRecognition() crea un ActivityTransitionRequest per monitorare entrata e uscita da WALKING, RUNNING, ON_BICYCLE, IN_VEHICLE. In caso di transizione, viene inviato un evento tramite un PendingIntent che attiva il TransitionReceiver e permette di avvisare con una notifica se l'utente è in movimento e ha voglia di iniziare un viaggio o meno.

I worker sono due, entrambi sono gestiti tramite WorkManager (libreria android che può essere vista come un sistema di gestione di task affidabile), sono InactivityWorker e ProgramTripWorker:

- **Inactivity Worker**, schedato ogni 24h, si occupa di segnalare all'utente se è da un periodo abbastanza lungo che non effettua un viaggio
- **ProgramTripWorker** invece si occupa di ricordare all'utente un viaggio programmato per la giornata odierna, per questo è schedato a mezzanotte.

5. Schermate e Funzionalità

Una volta introdotte le schermate in linea generale, gli obiettivi la struttura le funzionalità dell'applicazione, la struttura del db, il pattern utilizzato per la rappresentazione delle

schermate e aver anche introdotto Manifest e MainActivity, possiamo passare alla descrizione dettagliata delle schermate, tutte racchiuse nel package ui.

5.1 MAP

La schermata Map è la schermata principale dell'applicazione e rappresenta il punto di partenza per l'utente appena dopo l'avvio dell'app. Qui è possibile avviare e interrompere manualmente il tracciamento di un viaggio, monitorando in tempo reale il percorso tramite una linea tracciata sulla mappa (*Polyline*).

La schermata è gestita dal MapFragment, che implementa l'interfaccia OnMapReadyCallback per inizializzare la Google Map al caricamento. La logica principale è coordinata invece dal MapViewModel, che contiene i metodi per avviare, aggiornare e interrompere un viaggio (botone celeste con scritto Avvia/Stop), salvare i punti del percorso, aggiungere note e foto (Ovvero le due imageView che compaiono all'avvio di un viaggio) e aggiornare lo stato del tracciamento.

- **Avvio del tracciamento:**

L'utente può premere il pulsante "**Avvia**" per iniziare un nuovo viaggio. Al click viene eseguita la funzione `getCurrentPlace()`, che utilizza `FusedLocationProviderClient` per recuperare la posizione attuale dell'utente. Se la posizione è valida, si apre una finestra di dialogo (`selectTripTypeDialog`) che mostra l'elenco dei tipi di viaggio (Local, Excursion, Journey) definiti nell'enum `TripType`. Dopo la selezione, viene richiamato il metodo `startTrip()` del `MapViewModel`, che: crea un nuovo oggetto `Trip` con la posizione di partenza e il tipo selezionato. Inserisce il `Place` nel database (`placeRepository.insert()`), ne recupera l'ID con `getPlaceId()`, e registra la relazione `TripPlace` con il timestamp corrente. Salva l'ID del viaggio nella variabile `_tripId`. Infine, viene creato un Intent con l'azione `TrackingService.ACTION_START` e viene avviato il `TrackingService` in modalità foreground tramite `ContextCompat.startForegroundService()`. Prima di tutto questo flusso, il Fragment verifica che il permesso `ACCESS_FINE_LOCATION` sia già stato concesso (con `ActivityCompat.checkSelfPermission()`); in caso contrario, viene mostrato un Toast che invita l'utente ad abilitarlo manualmente.

- **Interruzione del tracciamento:**

premendo nuovamente il pulsante ("Stop"), il viaggio viene chiuso, viene registrata la posizione finale (come nel caso dell'avvio anche qui si usa `getCurrentPlace()`) e il tracciamento viene arrestato con `MapViewModel.stopTrip()`, che aggiorna la data di fine del viaggio (`tripRepository.updateTrip()`) e registra un `TripPlace` con la posizione finale. La linea sulla mappa scompare e lo stesso vale per le due image view con il simbolo "+" e la macchinetta fotografica, il cui scopo è quello di aggiungere foto e note.

- **Aggiornamento del percorso in tempo reale:**

Il MapFragment osserva il LiveData `tripPlaces` esposto dal `MapViewModel` (ottenuto con `placeRepository.getPlacedByIdTrip(_tripId)`), che contiene tutti i punti del

percorso. Quando l'elenco viene aggiornato, la lista `polylinePoints` viene ripopolata con i nuovi `LatLng`, e la `Polyline` associata alla mappa viene aggiornata in tempo reale

- **Aggiunta di fotografie:**

durante un viaggio attivo, l'utente può selezionare una foto dalla galleria(`Intent.ACTION_PICK`) cliccando sull'`image view` con al centro l'icona di una macchinetta fotografica. Il metodo `onActivityResult()` riceve l'immagine e, dopo aver verificato che esista un `Trip` attivo e almeno un `TripPlace`, salva la foto nello storage interno dell'applicazione e registra il percorso dell'immagine nel database tramite `addPhoto()` associandola all'ultimo punto registrato del percorso (o meglio l'ultimo place).

- **Aggiunta di note geolocalizzate:**

è possibile inserire note testuali tramite una finestra di dialogo che si apre quando l'utente clicca sull'`image view` con l'icona del “+” sopra a quella per aggiungere le foto. Ogni nota viene poi salvata con coordinate e timestamp.

Il sistema inoltre mantiene un flag (`tracking_running`) salvato nelle `SharedPreferences` per ricordare se un viaggio è attivo anche dopo il riavvio dell'app così che, all'avvio del `Fragment`, il metodo `refreshTrackingFlag()` aggiorna il `LiveData_trackingRunningFlag` (la variabile locale) e sincronizza l'interfaccia in base allo stato corrente.

5.2 PROGRAM TRIP

La schermata **ProgramTrip** consente di creare un piano di viaggio futuro, specificando i dettagli principali come le date, la tipologia, la descrizione e i luoghi di partenza e arrivo. La schermata è gestita dal `ProgramTripFragment`, mentre la logica principale di validazione e salvataggio è implementata nel `ProgramTripViewModel`, mentre `TripPlan` è l'entità di riferimento (model).

La schermata si presenta con uno `Spinner` per selezionare il tipo di viaggio e un form di edit text in cui inserire tutti i dati necessari per pianificare un viaggio: Data di partenza, Data di arrivo, Luogo di partenza (opzionale), Destinazione, Descrizione (opzionale) un bottone di salvataggio.

Il `ProgramTripFragment` ha solo il compito di recuperare tutti dati dal form e passarli ai metodi del view Model che si occuperanno della validazione (esempio: data di arrivo antecedente a quella di partenza oppure verifica se ci sono altri viaggi programmati già per il periodo specificato dall'utente). Sarà sempre responsabilità del view model anche salvare il piano di viaggio nel database e in caso di successo sarà lanciato anche un `Alert Dialog` di successo.

5.3 TRIP STORICAL

La schermata **Trips Storical** permette all'utente di consultare lo storico di tutti i viaggi effettuati. Da qui è possibile visualizzare le informazioni principali di ciascun viaggio (ad esempio tipologia, arco temporale, distanza percorsa) e accedere a due schermate secondarie dedicate: **Note** e **MapTrip**.

5.3.1 Trips Storical:

La schermata **Trips Storical** consente di consultare la lista di tutti i viaggi completati e di

accedere a ulteriori schermate di dettaglio relative a ciascun viaggio.

L'implementazione segue il pattern MVVM, garantendo una separazione netta tra: la gestione dei dati (TripViewModel), la logica di presentazione (TripAdapter e TripViewHolder), la visualizzazione (DescriptionFragment).

La struttura è composta da un RecyclerView che mostra l'elenco dei viaggi effettuati, un Spinner per filtrare i viaggi per tipologia (ALL, LOCAL, EXCURSION, JOURNEY) e un pulsante di filtro, mentre il layout utilizzato è nav_storical.xml.

Nel DescriptionFragment viene configurato lo Spinner, inizializzato il TripAdapter che sarà poi associato al RecyclerView (R.recyclerview_trips) e vengono osservati i viaggi registrati (registrati come LiveData e ottenuti con il metodo getAllTrips() del viewModel). Il metodo filtraViaggi() si occupa di richiamare il metodo del view model che applica i filtri alla lista di viaggi e anch'essa restituisce un LiveData. Oltre ai metodi di cui abbiamo già discusso nel TripsViewModel troviamo anche un metodo per ottenere la distanza percorsa in un viaggio, getDistanceById(), che prende in input l'id (univoco del viaggio). Quest'ultimo altro non fa che, data una lista di punti geografici (Place) registrati durante il viaggio, recuperati tramite il metodo getPlacedById(tripId), calcolare a coppie di punti consecutivi la distanza lineare tra di essi, con la funzione distanceBetween(), sommando le somme parziali in un'unica variabile e poi convertirla in km.

5.3.2 Note

La schermata **Notes** permette di visualizzare l'elenco di tutte le note create durante un viaggio selezionato e per accedere a questa schermata bisogna cliccare sul pulsante rosso Note presente nella card che rappresenta il viaggio (trip_card.xml). Nel nav_graph.xml è stata definita un'azione per questo passaggio di schermata. La schermata è gestita dal NotesFragment, mentre la logica di recupero e presentazione dei dati è implementata nel NoteViewModel. Il layout principale include un RecyclerView che mostra l'elenco ordinato delle note salvate e l'id del viaggio selezionato viene passato al fragment come argomento tramite un Bundle, al momento della navigazione dal TripAdapter.

All'avvio del NoteFragment l'id del viaggio selezionato (passato come argomento) viene recuperato. Successivamente vengono recuperati con l'id tutti i place con il metodo getPlacesById(tripId: Int) sotto forma di LiveData (quindi vengono osservati). Il RecyclerView poi userà il NoteAdapter per il popolamento mostrando una lista di note (un item è rappresentato da note_card.xml e gestito in NoteViewHolder) e recuperata dal NoteViewModel attraverso il metodo getNotesByTripId(tripId) del NoteRepository, esposta come LiveData e osservata nel NotesFragment per aggiornare la lista.

Per tornare alla schermata precedente questa volta non si utilizza la solita toolbar (è stata volontariamente nascosta), ma bensi basta cliccare su una imageView con l'icona di una freccia che va all'indietro. L'azione di questa imageView scatena requireActivity().onBackPressedDispatcher.onBackPressed() che farà tornare l'utente alla schermata di TripsStorical e ripristinerà la toolbar.

5.3.3 Map Trip

La schermata TripMap permette di visualizzare su mappa il percorso effettuato, tramite Polyline, durante un viaggio già concluso e anche questa schermata (come quella dedicata

alle note) è accessibile esclusivamente dalla lista dei viaggi presenti nella schermata Trips Historical, ma cliccando sulla card del viaggio selezionato. Il TripMapFragment implementa l'interfaccia OnMapReadyCallback per inizializzare e configurare la mappa al caricamento. A differenza delle altre schermate, qui non è stata creata una nuova classe ViewModel, ma è stato riutilizzato il TripViewModel già utilizzato per la schermata principale dei viaggi. Questa scelta è stata fatta perché la schermata ha una logica limitata e utilizza soltanto il metodo getPlacesById() per recuperare i punti del percorso. All'avvio del TripMapFragment, l'id del viaggio selezionato viene recuperato dal Bundle passato come argomento. La mappa è gestita tramite un SupportMapFragment. Quando è pronta (onMapReady()), viene osservato il LiveData restituito dal metodo getPlacesById(tripId). Questo metodo recupera dal repository tutti i punti (Place) registrati per il viaggio. Dopo un controllo per verificare che i punti siano effettivamente stati recuperati con successo (o che non si sia selezionato un viaggio vuoto) Tutti i punti vengono aggiunti alla polyline e disegnati sulla mappa. Come per la schermata dedicata alle note anche qui la toolbar scompare e l'unico modo per tornare alla schermata di Trips Storicaa e ripristinare la toolbar è cliccare sul bottone con scritto "back". Questa volta però viene usata un'altra action definita nel nav_graph.xml.

5.4 PHOTO

La schermata Galleria consente di visualizzare tutte le fotografie associate ai viaggi completati, organizzandole in gruppi distinti per ciascun viaggio (distinguibile per data di arrivo e partenza e città di arrivo e partenza). La schermata è gestita dal AddPhotoFragment, mentre la logica di recupero e presentazione dei dati è affidata al PhotoViewModel. Ogni gruppo mostra il titolo del viaggio e una lista orizzontale di fotografie. L'implementazione si basa su due RecyclerView annidati: uno principale che mostra l'elenco dei viaggi, e uno secondario che mostra le fotografie relative a ciascun viaggio.

All'avvio del AddPhotoFragment richiama il PhotoViewModel, che fornisce sia l'elenco dei viaggi conclusi tramite getAllTrips() sia i metodi per recuperare le fotografie associate a ciascun viaggio getPhotosByTrip(tripId: Int). Il fragment osserva il LiveData che contiene la lista dei viaggi. Quando i dati sono disponibili, viene lanciata una coroutine per filtrare i viaggi che hanno almeno una fotografia associata. Questo controllo avviene richiamando getCountPhotosByTrip(trip.id) per ciascun viaggio. La lista filtrata viene passata al GroupedPhotoAdapter, che aggiorna la visualizzazione. Essendo due recycler view annidati, uno per i viaggi e uno per le foto associate ad un singolo viaggio, vengono usate rispettivamente anche due adapter (GroupedPhotoAdapter e PhotoAdapter) e due view holder (GroupedViewHolder e PhotoViewHolder). I due view Holder gestiscono rispettivamente due xml uno per la singola foto (photo_place_card.xml) formata da un text per il time stamp e una image view e uno per le foto di un singolo viaggio (item_grouped_photo.xml) formata da un text view e da un recycler view di photo_place_card.

5.5 STATISTICS

La schermata Statistics consente di visualizzare in modo aggregato i dati relativi ai viaggi effettuati. L'obiettivo è offrire all'utente una panoramica completa tramite rappresentazioni grafiche e una mappa interattiva. La schermata è gestita dal StatisticsFragment, mentre la

logica di recupero, filtraggio e aggregazione dei dati è implementata nel `StatisticsViewModel`. Appena entrati nella pagina in altro troviamo subito uno spinner con 4 opzioni (classe `Filter_type` di tipo `enum`): `1_MONTH`, `3_MONTHS`, `8_MONTHS` e `ALL`. Questi 4 rappresentano i 4 tipi di filtro disponibili nel caso in cui l'utente volesse visionario solo le statistiche di un particolare periodo. Il conteggio dei mesi non si basa per 30 giorni, ma se ad esempio il 5 maggio avvio il filtro `1_MONTH` vedrò solo le statistiche innerenti ai viaggi effettuati dal 1 maggio al 5 maggio.

Le altre componenti che troviamo nella pagina sono:

1. Un `BarChart` che mostra il numero di viaggi effettuati per ciascun mese
2. Un `LineChart` che visualizza la distanza percorsa in ogni viaggio.
3. Una `Heatmap` per evidenziare le aree più frequentemente visitate.

Quando l'utente seleziona un filtro dallo `Spinner` e preme il pulsante di conferma, viene invocato `setFilter()` che aggiorna `_periodFilter`. Ogni volta che il filtro cambia, la `ViewModel` chiama `loadTripsForFilter()` e aggiorna il `LiveData` (garantendo aggiornamenti automatici della UI quando i dati cambiano) `filteredTrips`. Il `Fragment` osserverà i `filteredTrips` e aggiornerà i grafici e la heatmap.

5.6 MONUMENTS ALLARM

Questa schermata è dedicata alla gestione dei monumenti rilevanti per l'utente. Ogni monumento (`card_monument.xml`) contenente il nome e una checkbox che consente di attivare o disattivare il geofence associato. Quindi se il monumnto è checked allora riceveremo una notifica se entremo/usciremo da un' area circolare di 200m da dove quest' ultimo si trova. Le coordinate precise sono salvate nel db nella tabella `monuments`, coordinate ottenute tramite sistema di geocoder che entra in funzione quando viene aggiunto un nuovo monumento. Se ci sono problemi (es. luogo inesistente) sarà inviato un `Toast` di avvertimento. L'utente può cercare dinamicamente un monumento tramite una barra di ricerca e attivare il monitoraggio della sua posizione relativa al luogo/luoghi selezionato/i. Durante il salvataggio i geofence precedenti vengono rimossi e quelli nuovi vengono aggiunti in blocco come lista.

6. Tracking Service

Il `TrackingService` rappresenta il cuore del tracciamento GPS continuo all'interno dell'app. È implementato come un `Foreground Service`, necessario affinché il tracciamento rimanga attivo anche quando l'app è in background o lo schermo del dispositivo è spento, evitando che venga interrotto dal sistema operativo. Il servizio viene attivato o arrestato tramite `Intent` espliciti con azioni specifiche: `ACTION_START` e `ACTION_STOP`.

L'avvio del servizio avviene al momento della selezione del tipo di viaggio nel `MapFragment` e l'esecuzione è vincolata alla creazione di una notifica persistente. Questa notifica mostra all'utente che il tracciamento è attivo ed è fondamentale per mantenere il servizio in esecuzione in foreground.

Il servizio utilizza il `FusedLocationProviderClient` per ricevere aggiornamenti sulla posizione. Le frequenze e soglie di aggiornamento sono personalizzate in base alla tipologia di viaggio:

- LOCAL → aggiornamenti frequenti (ogni 20s, distanza $\geq 25\text{m}$)
- EXCURSION → ogni 30s, distanza $\geq 50\text{m}$
- JOURNEY → ogni 60s, distanza $\geq 150\text{m}$

Per ogni punto ricevuto, il servizio:

1. Verifica che la posizione sia recente e precisa.
2. Converte latitudine e longitudine in un oggetto Place, con nome del luogo ottenuto tramite Geocoder.
3. Salva il Place nel database e associa il punto al viaggio corrente con un oggetto TripPlace, tramite i rispettivi repository.

Inoltre, per i viaggi di tipo LOCAL ed EXCURSION, è stato implementato un controllo notturno (startMidnightChecker()): un processo che monitora l'orario e, alle 23:59, arresta automaticamente il tracking, registra l'ultima posizione disponibile (o una generica "Località sconosciuta" se il GPS è spento) e aggiorna la fine del viaggio.

L'arresto del servizio può avvenire manualmente tramite pulsante "Stop" nella mappa, oppure in modo automatico dal MidnightChecker. In entrambi i casi viene anche aggiornato il flag tracking_running salvato in SharedPreferences, così da mantenere coerenza allo stato dell'interfaccia anche in caso di riavvio dell'app

7. Worker (Notifications e background)

L'applicazione integra un sistema di notifiche che opera anche quando l'app non è in esecuzione in primo piano. Questo sistema è composto da due Worker periodici (programmati con WorkManager) e da un BroadcastReceiver che intercetta eventi relativi al movimento dell'utente (activity recognition). Le notifiche sono pensate per aumentare l'engagement e aiutare l'utente a non dimenticare viaggi programmati o occasioni di esplorazione.

7.1 InactivityWorker

La classe InactivityWorker estende Worker e viene schedulata ogni 24 ore per verificare se l'utente ha effettuato viaggi recentemente. Il suo obiettivo è quello di inviare una notifica motivazionale in caso di inattività prolungata. Innanzitutto Worker recupera l'ultimo viaggio completato (getMostRecentEndedTrip()) e confronta la data di destinazione con la data attuale. Se la differenza è superiore o uguale a 30 giorni viene inviata una notifica tramite NotificationManager (*"Ehi, è più di un mese che non fai un viaggio."*). Per testare il funzionamento della notifica ho provato ad inserire un arco temporale più corto (0 giorni), così da verificare se arrivasse o meno la notifica. La notifica è mostrata su un canale chiamato "Avvisi Viaggi", creato dinamicamente se non esistente.

7.2 ProgramTripWorker – Notifica di viaggio pianificato

Il ProgramTripWorker è un altro Worker periodico, schedato per l'esecuzione ogni giorno a mezzanotte. Il suo obiettivo è controllare se l'utente ha pianificato un viaggio per la giornata

corrente, e in tal caso inviare un promemoria. Recupera la data odierna (es. 2025-07-06) e chiama `getAllPlansForDate(today)` tramite `TripPlanRepository`.

Per ogni viaggio pianificato, costruisce un messaggio coerente con la tipologia (`TripType`) del piano e invia la notifica, che contiene anche un `PendingIntent` per aprire direttamente la `MainActivity`. Anche qui viene utilizzato il canale "Avvisi Viaggi". L'icona, il testo espanso (`BigTextStyle`) e il comportamento "click per aprire app" sono tutti configurati.

7.3 TransictionReceiver – Notifica di movimento rilevato

Il `TransictionReceiver` è un `BroadcastReceiver` registrato per intercettare eventi provenienti dall'`ActivityRecognitionClient`, tramite `ActivityTransitionRequest`. Ogni volta che viene rilevato un cambiamento di stato (es. l'utente inizia a camminare o a correre) il sistema invia un `Intent` che viene intercettato dal receiver. L'evento viene analizzato e se si tratta di un `ENTER` in attività (`WALKING`, `RUNNING`, `ON_BICYCLE` o `IN_VEHICLE`), viene generata una notifica in cui viene suggerito all'utente di iniziare a registrare un viaggio. La notifica è associata al canale "Monitoraggio attività". L'obiettivo dell'`ActivityRecognition` è quello di consentire all'app di proporre proattivamente la registrazione di un viaggio quando rileva un cambiamento contestuale nella mobilità dell'utente.

7.4 MonumentGeofenceReceiver

Per gestire gli eventi di ingresso e uscita dalle aree di interesse (vedi sezione 5.6), è stato implementato un `BroadcastReceiver` chiamato `MonumentGeofenceReceiver`.

Questo componente riceve l'intento generato dall'entrata o l'uscita da un'area riportata come geofence e mostra una notifica all'utente contenente il nome del luogo associato e se si sta entrando o uscendo dall'area.

